

TreeDiff: AST-Guided Code Generation with Diffusion-based LLMs (DLLMs)

Yiming Zeng^{1*} · Jinghan Cao^{2*} · Zexin Li³ · Yiming Chen^{&sup4;} · Tao Ren^{&sup5;} · Zhuochun Li^{&sup5;} · Dawei Xiang¹ · Xidong Wu^{&sup5;} · Shangqian Gao^{&sup6;} · Tingting Yu¹

¹University of Connecticut · ²San Francisco State University · ³UC Riverside · ^{&sup4;}National University of Singapore · ^{&sup5;}University of Pittsburgh · ^{&sup6;}Florida State University · * Equal contribution

🌀 **TreeDiff** = first AST-aware masking for diffusion code generation → **+13.3%** on HumanEval+, narrows gap with autoregressive models.

🕒 Motivation

Diffusion-based LLMs (DLLMs) struggle with **syntactic precision** in code generation. Standard random token masking:

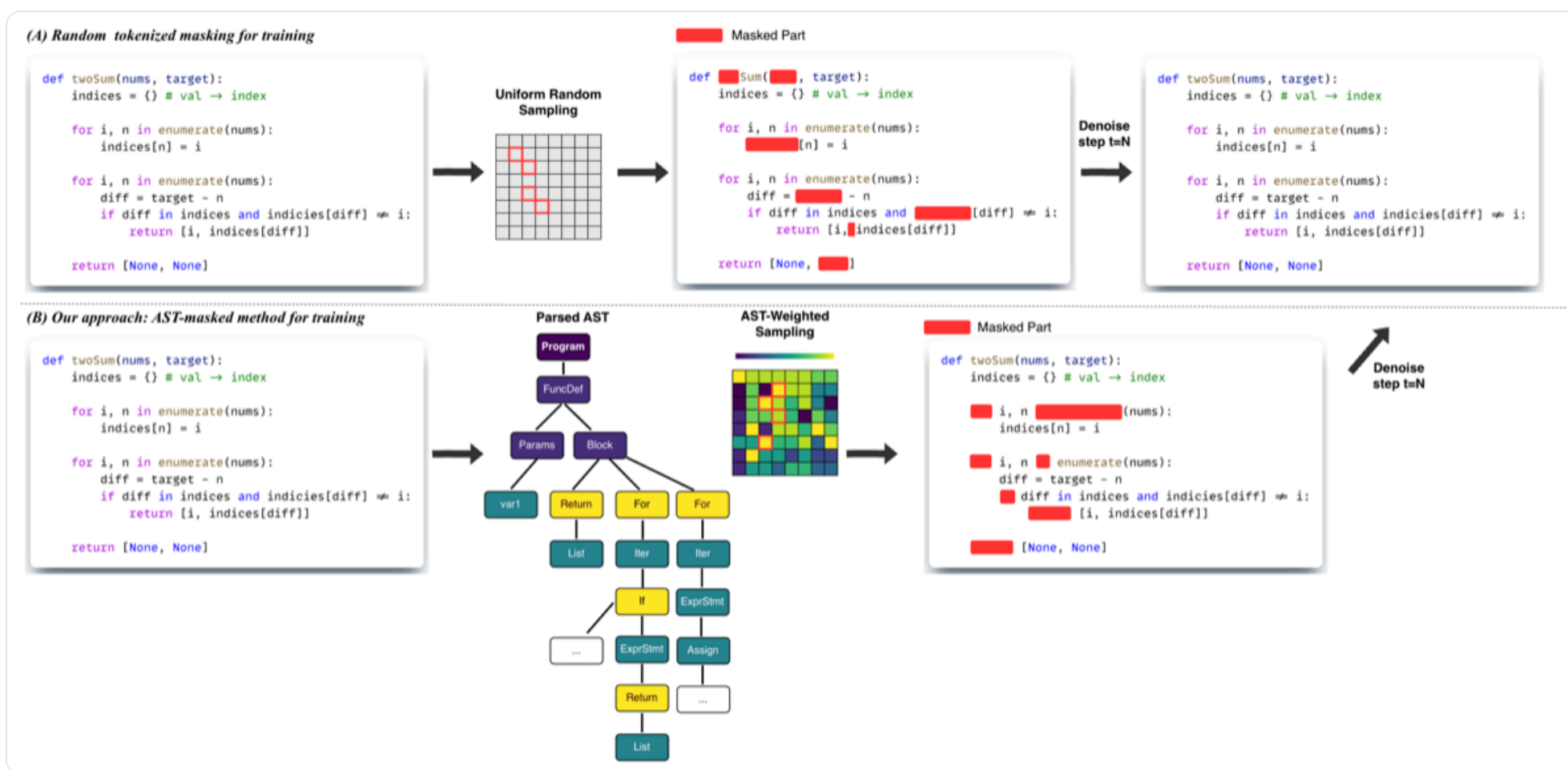
- Ignores syntactic boundaries during denoising
- Fails to capture long-range hierarchical dependencies
- Treats all tokens equally — but they're not!

🔑 CORE INSIGHT

“For code, noise should not be purely stochastic — structural priors enable DLLMs to learn program-level dependencies.”

🔗 Method: AST-Guided Masking

TreeDiff parses code into an **Abstract Syntax Tree (AST)** and uses it to guide which tokens to mask — targeting structurally critical nodes.



TreeDiff augments masked diffusion with **AST-aware policies**: the syntax tree guides which tokens to mask and how to weight the denoising loss.

📁 Region-Specific Masking

Each diff region gets a tailored masking policy:

Prompt
intact

Reasoning
random

Code
AST-weighted

⚖️ AST Node Weighting

Loss weights scale with AST node depth: **deeper nodes receive higher weight** than surface tokens.

$$w(\text{node}) = 1 + \alpha \cdot \text{depth}(\text{node})$$

Category	Examples	Weight	Purpose
Skeletal	Imports, Constants	0.15	Preserve skeleton
Data Flow	Assigns, Calls	0.42	Moderate priority
Conditionals	if, elif, else	0.58	Core logic
Control Flow	for, while, return	0.60	⚡ Highest priority

📊 Main Results (Pass@1 %)

+13.3%

Relative Gain
HumanEval+ (T=512)

42.1

HumanEval Pass@1
(vs 39.6 baseline)

Model	T = 256				T = 512			
	HE	HE+	MBPP	MBPP+	HE	HE+	MBPP	MBPP+
<i>Autoregressive Models†</i>								
DeepSee... Coder-33B	50.6	44.5	80.4	70.1	50.6	44.5	80.4	70.1
CodeQw... 7B	51.8	45.7	73.5	60.8	51.8	45.7	73.5	60.8
CodelLa... 13B	42.7	38.4	63.5	52.6	42.7	38.4	63.5	52.6
<i>Diffusion-based LLMs (LLaDA Backbone)</i>								
LLaDA-Instruct	39.6	34.8	51.9	43.1	36.6	31.7	39.4	31.7
+ Random Masking	39.6	35.4	52.9	43.9	38.4	32.3	41.5	32.8
<i>Ours</i>								
+ TreeDiff	42.1	37.2	52.9	44.2	40.2	36.6	42.3	33.3

† From EvalPlus Leaderboard. HE = HumanEval, HE+ = HumanEval+.

⚖️ Ablation: Masking Granularity

HumanEval		
Strategy	T=256	T=512
Random Masking	39.6 / 35.4	38.4 / 32.3
AST (Span)	40.9 / 36.6	40.2 / 36.6
AST (Node)	42.1 / 37.2	40.2 / 36.6
MBPP		
Strategy	T=256	T=512
Random Masking	52.9 / 43.9	41.5 / 32.8
AST (Span)	51.6 / 42.9	39.7 / 31.5
AST (Node)	52.9 / 44.9	42.3 / 33.3

💡 KEY INSIGHT

Random masking outperforms span masking on code diffs. Spans often split syntactic units (e.g., an if-condition), while random masking preserves more complete AST nodes. TreeDiff's token-level AST masking surgically targets high-influence nodes while preserving context.

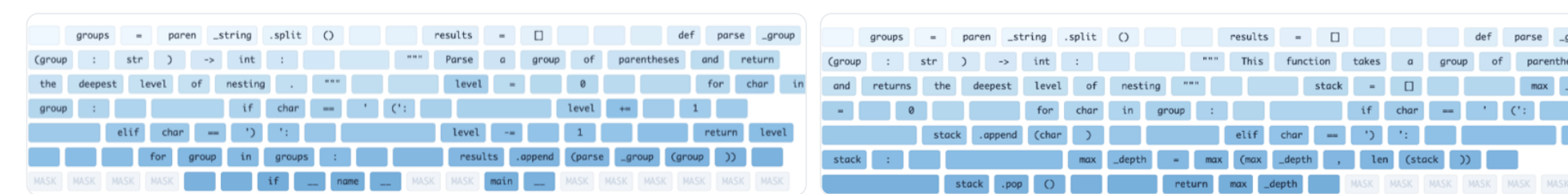
🛡️ Robustness Analysis

Method	T=512	T=1024
LLaDA + span mask	35.6	34.2
TreeDiff	42.1	41.8

TreeDiff maintains stable performance (−0.1%) as T increases 256→512→1024, while baselines degrade by ~2.1% due to fragmented syntax trees.

🕒 Generation Trajectory Analysis

Task: Max nesting depth of parentheses (HumanEval/6), Step 110/256.



Vanilla MD (Random Mask): Uses shallow counter heuristic, fails on nested structures. Fragmented generation.

TreeDiff: Establishes stack-based control flow early. Correctly captures recursive nesting.

Edit distance across denoising steps. Vanilla MD oscillates (54 → 52 → 41 → 46 → 32). TreeDiff converges monotonically with fewer steps.

Only 50 denoising steps (vs 256 decode steps) needed

<> Generated Code Comparison

HumanEval/6 — Nested parentheses depth:

Vanilla MD (Random Mask)	TreeDiff (AST-guided)
<pre>count = 0 for char in group: for char in group: if char == '(': count += 1 # Fails to track # nested peak depth</pre>	<pre>for char in group: if char == '(': stack.append(char) max_depth = max(max_depth, len(stack)) elif char == ')': if stack: stack.pop()</pre>

HumanEval/6: Maximum nesting depth of parentheses

☆ Key Contributions & Conclusion

- **First AST-aware masking** for DLLMs in code generation
- Novel AST-aware masking and loss weighting that amplifies structural signal
- Achieve **+13.3%** relative HumanEval+ and **42.1** Pass@1 with only 50 denoising steps — lower FLOPs than 256-step autoregressive decoding
- Practical recipe for adapting pretrained masked diffusion LMs to structured seq2seq tasks without catastrophic forgetting

TreeDiff addresses a fundamental limitation: **random masking ignores code structure**. By aligning corruption with AST hierarchy, the model learns programming language compositionality, reconstructing programs that respect grammatical boundaries.

